

MDS-001 — Design Token Architecture

External publication draft

MDS-001 — Design Token Architecture

Tokens are not values. They are the language through which the Design System communicates intent.

Purpose

The Mosaic Design Language (MDL) establishes:

- Why Mosaic exists.
- How decisions are made.
- How the platform thinks.
- How the platform behaves.
- How understanding is organised.

MDS-001 begins the implementation layer.

It defines how those concepts become a machine-readable design system.

Unlike many design systems, MDS-001 is **not** simply a colour token document.

It defines the architectural hierarchy through which every future Mosaic client will express the same conceptual experience.

Relationship to MDL

MDL

↓

Concepts

↓

Design Tokens

↓

Runtime Resolution

↓

Components

↓

Presentation

Tokens are the first implementation artefact.

Everything beneath MDS-001 depends upon them.

Scope

This specification defines:

- Token philosophy
- Token hierarchy
- Token taxonomy
- Semantic layers
- Runtime token model
- Token inheritance
- Token resolution
- Token lifecycle
- Versioning
- Extension interaction

This specification intentionally does **not** define:

- Colours
- Typography values
- Motion curves
- Components
- Materials

Those are defined by later MDS specifications.

Core Question

MDS-001 exists to answer one question.

How should design decisions become implementation?

Token Statement

Within Mosaic:

Tokens describe intent.

They do not describe implementation.

For example.

Poor.

Blue500

Better.

Brand.Primary

Better still.

Surface.Hero

The further implementation moves from raw values towards meaning, the easier the system becomes to evolve.

Primary Token Hierarchy

The Mosaic Design System intentionally separates tokens into conceptual layers.

Primitive

↓

Semantic

↓

Composition

↓

Component

↓

Runtime

↓

Presentation

Each layer has exactly one responsibility.

Future chapters define every layer in detail.

Expected Outcome

After reading MDS-001 contributors should understand:

- how tokens are organised
- why semantic tokens exist
- why runtime tokens exist
- how components consume tokens
- how extensions interact with tokens
- how future token systems should evolve

without discussing specific colour palettes or component implementations.

Repository Structure

design/

■■■ mds/

■■■ MDS-001 Design Token Architecture/

README.md

00-document-control.md

01-what-is-a-design-token.md

02-token-hierarchy.md

03-primitive-tokens.md

04-semantic-tokens.md

05-composition-tokens.md

06-runtime-tokens.md

07-token-resolution.md

08-token-inheritance.md

09-token-versioning.md

10-extension-tokens.md

11-governance.md

12-adrs.md

13-contributor-guidance.md

glossary.md

references.md

Dependencies

Required reading:

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model

Downstream specifications:

- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography
- MDS-005 Motion

- MDS-006 Composition Engine
- MDS-008 Component Library

Review Status

Status

Draft

Owner

Lead Design Systems Architect

Next File

00-document-control.md

Document Control

Document Information

Property	Value
Document ID	MDS-001
Title	Mosaic Design System — Design Token Architecture
Classification	Internal
Status	Draft
Version	0.1
Owner	Lead Design Systems Architect
Parent Specifications	MDL-001 → MDL-005
Repository	/design/mds/MDS-001 Design Token Architecture/

Purpose

MDS-001 defines the implementation architecture of the Mosaic Design System.

It establishes how abstract concepts defined within the MDL become machine-readable design decisions.

Unlike traditional token systems that primarily describe colours and spacing, Mosaic tokens represent **design intent**.

The Design Token Architecture therefore becomes the implementation bridge between:

- philosophy
- behaviour
- composition
- presentation

Authority

MDS-001 governs:

- Token hierarchy
- Token taxonomy
- Semantic naming
- Runtime resolution
- Token inheritance
- Token lifecycle
- Extension integration

- Cross-platform consistency

This specification intentionally does **not** define:

- colour palettes
- typography scales
- motion values
- materials
- components

Those specifications consume the architecture established here.

Relationship To MDL

The Design Token Architecture intentionally begins after every conceptual decision has already been made.

[mermaid]
flowchart TD

```
Vision
Vision --> Principles
Principles --> MentalModel
MentalModel --> Interaction
Interaction --> Composition
Composition --> Tokens
Tokens --> Presentation
```

Tokens should never redefine concepts established by MDL.

Their responsibility is implementation.

Not philosophy.

Design Intent

Most design systems expose implementation directly.

Examples include:

Blue500

8px

24px

Shadow3

These values communicate implementation.

They communicate almost nothing about meaning.

Mosaic intentionally separates:

Meaning

↓

Intent

↓

Implementation

Tokens therefore communicate:

Why a value exists

rather than merely:

What its value is.

Reader Expectations

Before reading this specification contributors should understand:

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model

MDS intentionally assumes that all conceptual work has already been completed.

Its responsibility is implementation.

Architectural Scope

The Design Token Architecture governs:

- token structure
- semantic intent
- runtime adaptation
- inheritance
- resolution
- stability
- portability

It intentionally avoids:

- framework APIs
- CSS variables
- Flutter themes
- SwiftUI implementations
- Compose implementations

Those become client-specific implementation details.

Stability

Expected lifetime.

Artefact	Expected Lifetime
CSS Variables	Months
Flutter Theme	Months
SwiftUI Theme	Months
Token Values	Years
Token Hierarchy	Years
Token Philosophy	Decades

The hierarchy should evolve significantly more slowly than the values it contains.

Success Criteria

MDS-001 succeeds when:

- every Mosaic client expresses the same design language
- token names communicate meaning rather than implementation
- runtime adaptation becomes implementation-independent
- contributors rarely consume primitive values directly
- future specifications naturally build upon this architecture

A successful Design Token Architecture should become almost invisible.

Contributors should think in semantic intent rather than implementation values.

Review Status

Status

Draft

Dependencies

- MDL-001
- MDL-002
- MDL-003
- MDL-004
- MDL-005

Supersedes

None.

Next File

What Is A Design Token?

Purpose

Before defining the Mosaic token architecture, contributors must first understand what a Design Token represents within the Mosaic Design System.

Many design systems describe tokens as variables.

Others describe them as design decisions.

Within Mosaic they are something slightly different.

A Design Token is **the implementation of design intent**.

This distinction underpins the remainder of MDS.

Definition

Within the Mosaic Design System, a **Design Token** is defined as:

The smallest implementation unit capable of expressing a design decision independently from any platform or technology.

A token should communicate:

- meaning
- intent
- responsibility

before it communicates:

- colour
- spacing
- typography
- implementation

Why Tokens Exist

Without tokens, implementation becomes tightly coupled to presentation.

Example.

Poor.

```
[css]
color: #4F46E5;
padding: 16px;
border-radius: 12px;
```

Nothing communicates:

- why these values exist
- whether they should change
- where they should be reused

Instead.

Surface.Primary

↓

Primitive Value

The meaning survives.

The value may evolve.

Tokens Are Not Variables

Although tokens often become:

- CSS variables
- Flutter ThemeData
- SwiftUI environment values
- Compose theme objects

they are **not** those things.

Those are implementation artefacts.

The token exists before implementation.

Token

↓

Platform Theme

↓

Runtime

↓

Rendering

This distinction allows Mosaic to remain implementation independent.

Tokens Express Intent

Every token should answer one question.

Why does this value exist?

Examples.

Poor.

Blue500

Better.

Brand.Primary

Better.

Surface.Hero.Background

Notice the progression.

Implementation becomes increasingly semantic.

Tokens Are Stable

Values change.

Intent changes far less frequently.

Example.

Brand.Primary

↓

Indigo

↓

Teal

↓

Cyan

The token remains.

Only its implementation changes.

Applications consuming semantic tokens therefore require no modification.

Tokens Are Hierarchical

Tokens intentionally exist within layers.

Primitive

↓

Semantic

↓

Composition

↓

Component

↓

Runtime

Each layer adds meaning.

No layer should bypass another without strong architectural justification.

Tokens Belong To The Design System

Applications should consume tokens.

Applications should **not** invent them.

This ensures:

- consistency
- maintainability
- portability
- accessibility

Every Mosaic client should therefore obtain design decisions from the same token architecture.

Tokens Are Cross-Platform

A Design Token should never depend upon:

- CSS
- Flutter
- SwiftUI
- Compose
- HTML

Instead.

`Surface.Primary`

↓

`CSS Variable`

↓

`Flutter Theme`

↓

`SwiftUI Environment`

↓

`Compose Theme`

The token remains identical.

Only implementation differs.

Tokens Are Machine Readable

Although tokens communicate design intent to people, they should also communicate structure to tooling.

Future tooling should be capable of generating:

- CSS variables
- JSON
- YAML
- Flutter
- SwiftUI
- Compose
- TypeScript
- Go

from one canonical token definition.

The architecture should therefore favour clarity over implementation convenience.

Tokens Are Runtime Aware

Unlike many design systems, Mosaic introduces Runtime Tokens.

Runtime Tokens allow the Design System to respond to:

- artwork
- Focus
- Context
- accessibility
- device class

while preserving the same semantic token hierarchy.

Example.

`Atmosphere.Primary`

↓

`Generated From Artwork`

The consuming component remains unaware of the generation process.

Tokens Are Not Components

One of the most common mistakes contributors make is treating components as tokens.

Example.

`Button.Primary`

This is generally **too specific**.

Instead.

Action.Primary

↓

Consumed By Button

The token communicates intent.

The component consumes that intent.

This separation significantly improves reuse.

Good Examples

Example 01

Text.Primary

Communicates meaning.

Not implementation.

Example 02

Surface.Hero

Communicates responsibility.

Not colour.

Example 03

Spacing.Section

Communicates relationship.

Not pixel values.

Anti-patterns

Colour Tokens As Semantic Tokens

Blue500

Meaning depends entirely upon colour.

Platform Tokens

CSS.Primary

Technology has leaked into architecture.

Component Tokens

CardBlueBackground

Multiple concepts merged into one token.

Reuse becomes difficult.

Magic Values

16

24

32

Values appear directly throughout implementation.

Meaning has been lost.

The Token Pipeline

Every Design Token should follow the same conceptual lifecycle.

```
[mermaid]
flowchart TD
```

```
Intent
Intent --> Token
Token --> PlatformTheme["Platform Theme"]
PlatformTheme["Platform Theme"] --> RuntimeResolution["Runtime Resolution"]
RuntimeResolution["Runtime Resolution"] --> Rendering
```

Notice that implementation appears only after the token has already communicated intent.

Relationship To Future Chapters

This chapter establishes what a token is.

The following chapters define:

- token hierarchy
- primitive tokens
- semantic tokens
- composition tokens
- runtime tokens
- token resolution

Together they form the complete Design Token Architecture.

Summary

Design Tokens are not implementation values.

They are the implementation of design intent.

Their responsibility is to preserve meaning while allowing implementation to evolve.

Every future Mosaic client should therefore think in tokens...

...not colours.

...not spacing.

...not components.

Understanding should always precede implementation.

Review Status

Status

Draft

Next File

02-token-hierarchy.md

Token Hierarchy

Purpose

A Design Token Architecture is only valuable if every token has a clearly defined responsibility.

Without hierarchy:

- tokens duplicate one another
- components consume inconsistent values
- themes become difficult to evolve
- runtime adaptation becomes unpredictable

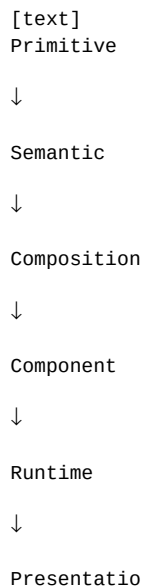
This chapter defines the canonical hierarchy of every token used within Mosaic.

Every future MDS specification should build upon this hierarchy.

It should never replace it.

The Hierarchy

The Mosaic Design System intentionally separates tokens into six conceptual layers.

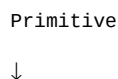


Each layer exists for one reason only.

No layer should perform the responsibilities of another.

Why Layers Exist

Most token systems contain only two layers.



Semantic

This works well for relatively static interfaces.

Mosaic intentionally supports:

- adaptive composition
- runtime atmosphere
- contextual hierarchy
- artwork-driven experiences
- multiple client platforms

These capabilities require additional abstraction.

The hierarchy therefore separates:

Meaning

from

Implementation

at every stage.

Layer 01

Primitive

Purpose:

Represent physical values.

Primitive Tokens answer:

What physical value exists?

Examples include:

- colours
- spacing
- typography sizes
- blur radii
- corner radii
- elevation values

Primitive Tokens intentionally possess **no semantic meaning**.

Example.

`Primitive.Colour.Indigo.500`

Nothing about this token explains:

- where it should be used

- why it exists

That responsibility belongs to Semantic Tokens.

Layer 02

Semantic

Purpose:

Represent design intent.

Semantic Tokens answer:

Why does this value exist?

Examples include:

Surface.Primary

Text.Secondary

Border.Subtle

Action.Primary

Atmosphere.Supporting

Components should consume Semantic Tokens rather than Primitive Tokens.

This allows implementation to evolve without changing consuming code.

Layer 03

Composition

Purpose:

Represent compositional intent.

Composition Tokens answer:

What role does this concept play within the current Composition?

Examples include:

Hero

Anchor

Supporting

Peripheral

Overlay

Navigation

Composition Tokens intentionally describe:

importance

rather than
appearance.

Layer 04

Component

Purpose:

Map semantic meaning onto reusable interface primitives.

Component Tokens answer:

How should this component express its role?

Examples include:

Button.Primary

Tile.Hero

Navigation.Active

Timeline.Current

Component Tokens should consume:

- Semantic Tokens
- Composition Tokens

They should never consume Primitive Tokens directly.

Layer 05

Runtime

Purpose:

Adapt the Design System to the user's current World.

Runtime Tokens answer:

What changed?

Examples include:

Atmosphere.Primary

Current.Domain

Current.Focus

Device.Class

Accessibility.Motion

Runtime Tokens are generated.

They are not manually authored.

Future runtime systems will continuously resolve these tokens.

Layer 06

Presentation

Purpose:

Produce implementation artefacts.

Examples include:

- CSS Variables
- Flutter ThemeData
- SwiftUI Environment
- Compose Theme
- Design Kit Variables

Presentation is intentionally excluded from the Design Token Architecture.

It is generated from it.

Responsibility Matrix

Layer	Responsibility	Changes Frequently
Primitive	Physical values	Rarely
Semantic	Meaning	Very rarely
Composition	Experience roles	Rarely
Component	Interface mapping	Occasionally
Runtime	Current World	Continuously
Presentation	Platform implementation	Continuously

Notice that the higher layers change significantly less frequently than the lower layers.

Information Flow

Token flow always proceeds in one direction.

```
[mermaid]
flowchart TD
```

```
Primitive
Primitive --> Semantic
Semantic --> Composition
Composition --> Component
Component --> Runtime
Runtime --> Presentation
```

Tokens should never resolve backwards.

For example:

Presentation should never determine:

- Semantic
- Composition
- Priority

Understanding always precedes implementation.

Dependency Rules

Every layer may consume tokens from layers above it.

Example.

Component

↓

Semantic

↓

Primitive

It may **not** bypass layers unnecessarily.

Poor.

Component

↓

Primitive

Meaning has been skipped.

This weakens maintainability.

Hierarchy Example

A Hero Tile.

[text]

Primitive

↓

Colour.Indigo.600

↓

Semantic

↓

Surface.Primary

↓

Composition

↓

Hero

↓

Component

↓

Tile.Hero

↓

Runtime

↓

Atmosphere.Primary

↓

Presentation

↓

CSS Variable

Every layer contributes exactly one responsibility.

No layer duplicates another.

Layer Independence

Each layer should remain independently replaceable.

Examples.

Primitive colours may change.

Semantic meaning remains.

Component implementations may change.

Composition remains.

Presentation frameworks may change.

Runtime behaviour remains.

This separation significantly improves long-term maintainability.

Anti-patterns

Component → Primitive

Components directly consuming physical values.

Meaning disappears.

Runtime → Primitive

Runtime generating raw colours instead of semantic intent.

Consistency weakens.

Semantic → Presentation

Semantic Tokens directly referencing CSS variables.

Architecture becomes platform dependent.

Composition As Components

Treating:

Hero

as a component rather than a compositional role.

Future adaptability decreases.

Hierarchy Litmus Test

Contributors should be able to answer:

Which responsibility belongs exclusively to this layer?

If the answer is:

"Several."

The hierarchy probably requires refinement.

Every layer should possess one responsibility.

Relationship To Future Specifications

Future specifications define every layer in greater detail.

Examples include:

- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography
- MDS-006 Composition Engine
- MDS-008 Component Library

Each specification should extend one or more layers without redefining the hierarchy itself.

Summary

The Token Hierarchy is the backbone of the Mosaic Design System.

It ensures that:

- meaning remains stable
- implementation remains flexible
- runtime adaptation becomes possible
- clients remain consistent
- contributors think in intent rather than values

Every future token introduced into Mosaic should have a clear home within this hierarchy.

Review Status

Status

Draft

Next File

03-primitive-tokens.md

Primitive Tokens

Purpose

Primitive Tokens form the physical foundation of the Mosaic Design System.

They represent measurable values rather than design intent.

Primitive Tokens intentionally possess **no semantic meaning**.

This distinction is fundamental.

Primitive Tokens describe **what exists physically**.

They never describe **why it exists**.

Definition

Within MDS, a **Primitive Token** is defined as:

A platform-independent physical design value that contains no contextual or semantic meaning.

Primitive Tokens are the lowest stable layer within the Design Token Architecture.

Every higher token layer ultimately resolves to Primitive Tokens.

Applications should almost never consume them directly.

Why Primitive Tokens Exist

Every design system ultimately resolves to physical values.

Examples include:

- colours
- spacing
- typography sizes
- corner radii
- opacity
- blur
- elevation

Without Primitive Tokens, these values become duplicated throughout implementations.

Primitive Tokens establish one canonical source of physical truth.

Primitive Tokens Are Value Objects

Primitive Tokens intentionally communicate only measurable values.

Examples.

Good.

`Primitive.Colour.Indigo.500`

`Primitive.Space.16`

`Primitive.Radius.8`

`Primitive.Blur.24`

Poor.

`PrimaryButtonBlue`

`HeroSpacing`

`SidebarRadius`

These contain implementation meaning.

Primitive Tokens must remain implementation agnostic.

Primitive Tokens Are Not Semantic

One of the most important architectural rules within Mosaic is:

Primitive Tokens never communicate intent.

Example.

`Primitive.Colour.Blue.500`

This token does **not** mean:

- primary
- action
- hero
- navigation

It simply describes one physical colour.

Semantic meaning belongs to the next layer.

Primitive Categories

The Primitive layer is intentionally small.

Current categories include:

Primitive

■■■ Colour

■■■ Space

■■■ Radius

■■■ Typography

■■■ Elevation

■■■ Blur

■■■ Opacity

■■■ Duration

■■■ Easing

■■■ ZIndex

Future categories should be introduced sparingly.

Primitive growth should remain slow.

Colour

Purpose.

Represent raw colour values.

Examples.

`Primitive.Colour.Cyan.500`

`Primitive.Colour.Slate.900`

`Primitive.Colour.White`

`Primitive.Colour.Black`

Primitive Colours should never be consumed directly by components.

Space

Purpose.

Represent physical spacing units.

Examples.

`Primitive.Space.2`

`Primitive.Space.4`

`Primitive.Space.8`

`Primitive.Space.16`

`Primitive.Space.24`

`Primitive.Space.32`

Spacing meaning belongs to Semantic Tokens.

Radius

Purpose.

Represent physical corner radii.

Examples.

`Primitive.Radius.4`

`Primitive.Radius.8`

`Primitive.Radius.12`

`Primitive.Radius.20`

Radius Tokens intentionally avoid describing where they should be used.

Typography

Purpose.

Represent measurable typography values.

Examples.

`Primitive.Font.Size.14`

`Primitive.Font.Weight.600`

`Primitive.LineHeight.24`

Typography hierarchy belongs to later specifications.

Primitive Tokens only communicate measurable values.

Elevation

Purpose.

Represent physical depth.

Examples.

`Primitive.Elevation.0`

`Primitive.Elevation.1`

`Primitive.Elevation.2`

Whether elevation communicates:

- Hero
- Overlay
- Surface

is determined later.

Blur

Purpose.

Represent physical blur intensity.

Examples.

`Primitive.Blur.8`

`Primitive.Blur.16`

`Primitive.Blur.24`

Blur meaning belongs to Material Tokens.

Primitive Blur simply defines available values.

Opacity

Purpose.

Represent transparency.

Examples.

`Primitive.Opacity.100`

`Primitive.Opacity.80`

`Primitive.Opacity.60`

`Primitive.Opacity.40`

These values intentionally possess no semantic meaning.

Motion

Primitive Motion consists of measurable values only.

Examples.

`Primitive.Duration.100`

`Primitive.Duration.200`

`Primitive.Duration.300`

`Primitive.Easing.Standard`

`Primitive.Easing.Decelerate`

Motion intent belongs to later specifications.

Primitive Naming

Primitive Tokens should follow the same naming convention.

Primitive

↓

Category

↓

Family

↓

Variant

Example.

Primitive

↓

Colour

↓

Cyan

↓

500

Naming should communicate structure.

Not usage.

Primitive Stability

Primitive Tokens should change infrequently.

Changing Primitive values potentially affects every consuming Semantic Token.

Consequently:

Primitive additions are preferred over frequent modification.

Existing Primitive Tokens should remain stable whenever practical.

Primitive Consumption

Applications should avoid consuming Primitive Tokens directly.

Poor.

Button

↓

Primitive.Colour.Cyan.500

Preferred.

Button

↓

Semantic.Action.Primary

↓

Primitive.Colour.Cyan.500

Meaning remains preserved.

Primitive Independence

Primitive Tokens should remain:

- platform independent
- framework independent
- component independent

Primitive Tokens should never reference:

- CSS
- Flutter
- SwiftUI
- HTML
- Components

Implementation belongs to later layers.

Anti-patterns

Semantic Primitive

`Primitive.Primary`

Meaning has leaked into the Primitive layer.

Component Primitive

`Primitive.Button.Blue`

Component responsibility has leaked downwards.

Runtime Primitive

`Primitive.CurrentArtwork`

Runtime behaviour belongs elsewhere.

Platform Primitive

`Primitive.CSS.Blue`

Platform concerns should never appear within Primitive Tokens.

Primitive Model

[mermaid]
flowchart TD

```
Primitive
Primitive --> Semantic
Semantic --> Composition
Composition --> Component
Component --> Runtime
Runtime --> Presentation
```

Primitive Tokens provide the physical foundation.

Every higher layer progressively adds meaning.

Litmus Test

A contributor should be able to ask:

Could this value exist without Mosaic?

If the answer is yes...

It probably belongs within Primitive Tokens.

If the answer depends upon:

- context
- behaviour
- hierarchy
- interface

it belongs elsewhere.

Summary

Primitive Tokens intentionally know nothing about the product.

They simply provide stable physical values.

Everything meaningful within Mosaic emerges by progressively layering:

- semantics
- composition
- components
- runtime behaviour

on top of these primitive foundations.

That separation is what allows the Design System to evolve without losing conceptual integrity.

Review Status

Status

Draft

Next File

04-semantic-tokens.md

Semantic Tokens

Purpose

Primitive Tokens describe physical values.

Semantic Tokens describe **meaning**.

This distinction is the single most important architectural separation within the Mosaic Design System.

If Primitive Tokens answer:

"What value exists?"

Semantic Tokens answer:

"Why does this value exist?"

Applications should think almost exclusively in Semantic Tokens.

Primitive Tokens should rarely appear outside the Design System itself.

Definition

Within MDS, a **Semantic Token** is defined as:

A platform-independent design decision expressed in terms of meaning rather than physical implementation.

Semantic Tokens deliberately avoid describing:

- colour
- spacing
- radius
- typography
- blur

Instead they describe:

- purpose
- responsibility
- role

Why Semantic Tokens Exist

Imagine the following implementation.

```
[css]
.button {
  background: #06b6d4;
}
```

This immediately creates several problems.

- Why cyan?
- Can it change?
- Should every button use it?
- Does another component depend on it?

Now compare.

```
[text]  
Action.Primary
```

The implementation becomes:

```
Action.Primary
```

↓

```
Primitive.Colour.Cyan.500
```

The reason survives.

The value may evolve.

Semantic Tokens Are Design Decisions

A Semantic Token represents a deliberate design decision.

Examples include:

```
Text.Primary
```

```
Surface.Canvas
```

```
Action.Primary
```

```
Border.Subtle
```

```
Status.Success
```

Each communicates:

- purpose
- intention
- responsibility

rather than implementation.

Semantic Categories

The Mosaic Design System currently defines the following semantic categories.

Semantic

■■■ Brand

■■■ Surface

■■■ Text

■■■ Border

■■■ Icon

■■■ Action

■■■ Status

■■■ Material

■■■ Atmosphere

■■■ Elevation

■■■ Motion

■■■ Focus

Future categories should remain intentionally limited.

Semantic Tokens describe concepts.

Not features.

Brand

Purpose.

Communicate Mosaic identity.

Examples.

Brand.Primary

Brand.Secondary

Brand.Accent

Brand Tokens should remain stable regardless of:

- artwork
- runtime
- themes

Brand communicates Mosaic.

Not media.

Surface

Purpose.

Describe compositional surfaces.

Examples.

Surface.Canvas

Surface.Primary

Surface.Secondary

`Surface.Overlay`

`Surface.Hero`

Notice that these describe **roles**.

Not colours.

Text

Purpose.

Communicate reading hierarchy.

Examples.

`Text.Primary`

`Text.Secondary`

`Text.Tertiary`

`Text.Disabled`

`Text.Inverse`

Text Tokens describe information hierarchy.

Not typography values.

Border

Purpose.

Communicate separation.

Examples.

`Border.Subtle`

`Border.Strong`

`Border.Focus`

The implementation may change.

The conceptual role remains stable.

Action

Purpose.

Communicate user intent.

Examples.

`Action.Primary`

`Action.Secondary`

`Action.Destructive`

Action.Passive

Buttons consume Action Tokens.

The tokens do not belong to buttons.

This significantly improves reuse.

Status

Purpose.

Communicate state.

Examples.

Status.Success

Status.Warning

Status.Error

Status.Information

Status Tokens intentionally avoid implementation colours.

Accessibility and presentation are resolved later.

Material

Purpose.

Describe the conceptual role of physical materials.

Examples.

Material.Canvas

Material.Acrylic

Material.Hero

Material.Overlay

Material Tokens intentionally avoid:

- blur values
- opacity
- colours

Those belong to Primitive Tokens.

Atmosphere

Purpose.

Communicate artwork-derived environmental intent.

Examples.

`Atmosphere.Primary`

`Atmosphere.Secondary`

`Atmosphere.Supporting`

Unlike Brand Tokens...

Atmosphere Tokens are expected to resolve differently at runtime.

Focus

Purpose.

Communicate emphasis.

Examples.

`Focus.Primary`

`Focus.Secondary`

`Focus.Background`

These Tokens should influence:

- hierarchy
- composition
- emphasis

rather than presentation directly.

Semantic Tokens Never Reference Components

Incorrect.

`Button.Primary`

`Card.Background`

`Sidebar.Border`

Correct.

`Action.Primary`

`Surface.Primary`

`Border.Subtle`

Components consume Semantic Tokens.

Semantic Tokens should never know components exist.

Semantic Tokens Consume Primitive Tokens

Example.

`Semantic.Surface.Primary`

↓

`Primitive.Colour.Slate.900`

Future themes may resolve the same Semantic Token differently.

Consumers remain unchanged.

Runtime Independence

Semantic Tokens intentionally remain independent from runtime.

Incorrect.

`Surface.CurrentArtwork`

Correct.

`Surface.Hero`

↓

`Runtime.Atmosphere.Primary`

Runtime adapts implementation.

Semantic meaning remains stable.

Naming Convention

Semantic Tokens should follow the same hierarchy.

Category

↓

Role

↓

Variant

Examples.

`Text.Primary`

`Surface.Canvas`

`Action.Destructive`

`Status.Warning`

Naming should always communicate meaning.

Never implementation.

Semantic Stability

Semantic Tokens should change significantly less frequently than Primitive Tokens.

Expected lifetime.

Layer	Lifetime
Primitive Values	Years
Semantic Meaning	Many Years

Changing Semantic Tokens potentially affects the conceptual language of the Design System.

Such changes should therefore be rare.

Good Examples

Surface.Canvas

↓

Primitive.Colour.Slate.950

Action.Primary

↓

Primitive.Colour.Cyan.500

Text.Secondary

↓

Primitive.Colour.Slate.400

Meaning remains completely independent from implementation.

Anti-patterns

Component Semantics

Button.Blue

Meaning depends upon implementation.

Colour Semantics

Blue.Primary

Meaning depends upon colour.

Platform Semantics

CSS.Primary

Architecture becomes implementation dependent.

Runtime Semantics

Artwork.Background

Runtime concepts belong elsewhere.

Semantic Model

[mermaid]
flowchart TD

```
Primitive
Primitive --> Semantic
Semantic --> Composition
Composition --> Component
Component --> Runtime
Runtime --> Presentation
```

Semantic Tokens introduce meaning.

Everything above them should reason about that meaning.

Everything below them should implement it.

Litmus Test

Contributors should ask:

If every Primitive value changed tomorrow, would this token still make sense?

If the answer is yes...

It probably belongs within the Semantic layer.

If the answer is no...

It probably belongs within Primitive Tokens instead.

Summary

Semantic Tokens represent the language of the Design System.

They intentionally separate:

meaning

from

implementation.

This separation allows Mosaic to:

- evolve themes
- support runtime adaptation
- preserve accessibility
- maintain consistency
- remain implementation independent

Every future MDS specification should consume Semantic Tokens rather than physical values whenever practical.

Review Status

Status

Draft

Next File

05-composition-tokens.md

Composition Tokens

Purpose

Primitive Tokens describe physical values.

Semantic Tokens describe design intent.

Composition Tokens describe **compositional roles**.

This distinction is unique to the Mosaic Design System.

Traditional design systems frequently stop at Semantic Tokens.

Mosaic introduces an additional layer because the platform's interface is not static.

It is continuously reorganised by the Composition Model.

Composition Tokens provide the implementation bridge between:

- MDL-005 Composition Model

and

- runtime presentation.

Definition

Within MDS, a **Composition Token** is defined as:

A token describing the compositional responsibility of an element independently from its presentation.

Composition Tokens answer:

"What role does this thing play inside the current Composition?"

They intentionally avoid answering:

"What should it look like?"

Why Composition Tokens Exist

Imagine two pieces of information.

Continue Watching

↓

Next Episode

Both may consume:

Surface.Primary

Both may consume:

Text.Primary

Yet one is significantly more important.

Semantic Tokens cannot express this difference.

Composition Tokens can.

Composition Before Components

Composition Tokens intentionally exist before Components.

Composition

↓

Component

↓

Presentation

This allows:

- Hero
- Timeline
- Navigation
- Progress

to share identical compositional behaviour while using completely different components.

Composition Categories

The current Composition Token taxonomy consists of:

Composition

■■■ Hero

■■■ Anchor

■■■ Primary

■■■ Supporting

■■■ Contextual

■■■ Peripheral

■■■ Overlay

■■■ Navigation

■■■ Modal

■■■ Background

These categories are conceptual.

They intentionally avoid implementation terminology.

Hero

Purpose.

Communicate the primary concept within the current Composition.

Example.

`Composition.Hero`

The Hero token should influence:

- emphasis
- visual weight
- breathing space
- movement priority

It should not define:

- colour
- typography
- dimensions

Those responsibilities belong elsewhere.

Anchor

Purpose.

Preserve orientation.

Example.

`Composition.Anchor`

Anchors intentionally remain behaviourally stable.

Future runtime systems should preferentially preserve Anchors during adaptive recomposition.

Primary

Purpose.

Represent information directly supporting the Hero.

Examples include:

- progress
- next episode
- current chapter

Primary Tokens communicate conceptual proximity to the Hero.

Supporting

Purpose.

Strengthen understanding without competing for attention.

Examples include:

- timeline
- queue
- continuation
- bookmarks

Supporting Tokens should remain visible while allowing the Hero to dominate.

Contextual

Purpose.

Provide explanation.

Examples include:

- cast
- author
- soundtrack
- reviews
- relationships

Contextual Tokens enrich understanding.

They should rarely become the visual centre of a Composition.

Peripheral

Purpose.

Represent information valuable but not immediately relevant.

Examples include:

- collections
- history
- statistics
- diagnostics

Peripheral Tokens should naturally compress before higher-priority compositional roles.

Overlay

Purpose.

Temporarily communicate information requiring immediate interaction.

Examples include:

- playback controls
- search
- confirmations

Overlay Tokens intentionally possess temporary behavioural ownership.

They should relinquish emphasis once interaction completes.

Navigation

Purpose.

Represent navigational anchors.

Unlike Hero Tokens...

Navigation Tokens should optimise:

- stability
- predictability
- orientation

rather than attention.

Background

Purpose.

Communicate supporting environmental information.

Examples include:

- atmospheric materials
- decorative relationships
- passive artwork

Background Tokens should never compete with active understanding.

Composition Tokens Are Behavioural

Unlike Semantic Tokens, Composition Tokens intentionally change.

Example.

Timeline

↓

Supporting

After playback completes.

Timeline

↓

Primary

Nothing about the Timeline changed.

Only its compositional role.

Composition Tokens therefore describe **behavioural intent** rather than permanent identity.

Composition Tokens Consume Semantic Tokens

Example.

Composition.Hero

↓

Semantic.Surface.Hero

↓

Primitive Values

Composition Tokens should never reference Primitive Tokens directly.

Meaning should remain layered.

Runtime Adaptation

Composition Tokens are expected to participate heavily in runtime adaptation.

Example.

Composition.Supporting

↓

Composition.Primary

This promotion occurs because:

- Context changed
- Priority changed
- Behaviour changed

The component itself remains identical.

Only its compositional responsibility changes.

Component Consumption

Components should consume Composition Tokens.

Not infer them.

Poor.

Timeline

↓

If Important...

Preferred.

Composition.Primary

↓

Timeline

The Composition Engine owns compositional reasoning.

Components communicate it.

Composition Independence

Composition Tokens intentionally remain independent from:

- devices
- layouts
- frameworks
- rendering engines

Desktop.

Composition.Hero

Television.

Composition.Hero

Voice.

Composition.Hero

The role remains identical.

Presentation changes.

Good Examples

Composition.Hero

↓

Surface.Hero

↓

Tile.Hero

Composition.Supporting

↓

Surface.Secondary

↓

Timeline

Composition.Peripheral

↓

Surface.Canvas

↓

Collection Shelf

Each layer contributes one responsibility.

Anti-patterns

Component Composition

Timeline.Primary

Component owns composition.

Incorrect.

Colour Composition

BlueHero

Presentation has leaked into composition.

Device Composition

MobileHero

Composition should remain device independent.

Static Composition

Composition Tokens never change.

Adaptive behaviour becomes impossible.

Composition Token Model

[mermaid]
flowchart TD

```
Semantic
Semantic --> Composition
Composition --> Component
Component --> Runtime
Runtime --> Presentation
```

Composition Tokens bridge:

design intent

and

runtime behaviour.

Litmus Test

Contributors should ask:

If this component disappeared tomorrow, would this compositional role still exist?

If the answer is yes...

It belongs within Composition Tokens.

If the answer is no...

It probably belongs within Component Tokens instead.

Summary

Composition Tokens are one of the defining innovations of the Mosaic Design System.

They allow runtime systems to reorganise understanding without requiring components to understand behaviour.

This separation enables:

- adaptive composition
- runtime hierarchy
- device independence
- plugin consistency
- future evolution

Every future runtime implementation should treat Composition Tokens as behavioural intent rather than visual styling.

Review Status

Status

Draft

Next File

06-runtime-tokens.md

Runtime Tokens

Purpose

Runtime Tokens are the most distinctive layer of the Mosaic Design System.

Unlike Primitive, Semantic and Composition Tokens, Runtime Tokens are **not authored**.

They are **resolved**.

Their purpose is to allow the Design System to respond intelligently to the user's current World while preserving the architectural guarantees established by the MDL.

Runtime Tokens enable Mosaic to feel alive without sacrificing consistency.

Definition

Within MDS, a **Runtime Token** is defined as:

A dynamically resolved design token whose value is derived from the user's current World at render time.

Runtime Tokens are generated.

They are never hardcoded.

They represent the intersection between:

- the user's World
- the current Composition
- device capabilities
- accessibility
- runtime environment

Why Runtime Tokens Exist

Traditional Design Systems assume:

Theme

↓

Static Values

↓

Interface

This approach works well for productivity software.

Mosaic is different.

The interface should subtly reflect:

- current artwork
- current Focus
- current Domain
- current atmosphere
- accessibility
- platform capabilities

without changing the underlying Design Language.

Runtime Tokens provide this capability.

Runtime Does Not Replace Design

One misconception must be avoided.

Runtime Tokens **never redefine the Design System**.

Instead they resolve existing design intent.

Incorrect.

Artwork

↓

Entire Theme Changes

Correct.

Artwork

↓

Atmosphere Tokens

↓

Semantic Tokens

↓

Existing Components

The architecture remains stable.

Only values evolve.

Runtime Inputs

Runtime Tokens may consume information from many sources.

Examples include:

[text]

Current World

↓

Current Focus

↓

Current Context

↓

Artwork

↓

Accessibility

↓

Device Class

↓

System Theme

↓

User Preferences

Importantly...

No single input possesses authority.

The Runtime Resolver evaluates them collectively.

Runtime Categories

Current Runtime Token categories include:

[text]
Runtime

■■■ Atmosphere

■■■ Accessibility

■■■ Device

■■■ Focus

■■■ Context

■■■ Motion

■■■ Elevation

■■■ Environment

■■■ Theme

Future Runtime categories should remain intentionally small.

Runtime complexity should remain hidden from component authors.

Atmosphere

Atmosphere Tokens communicate the emotional characteristics of the current World.

Examples.

`Runtime.Atmosphere.Primary`

`Runtime.Atmosphere.Secondary`

`Runtime.Atmosphere.Highlight`

These tokens are typically influenced by:

- artwork
- colour extraction
- luminance
- composition

Atmosphere enhances.

It never dominates.

Focus

Focus Tokens communicate the current conceptual emphasis.

Examples.

`Runtime.Focus.Primary`

`Runtime.Focus.Supporting`

`Runtime.Focus.Background`

These values influence:

- hierarchy
- emphasis
- movement
- composition

They should not redefine behaviour.

Context

Context Tokens adapt the presentation to the user's current activity.

Examples.

`Runtime.Context.Playback`

`Runtime.Context.Reading`

`Runtime.Context.Exploring`

The same Semantic Token may resolve differently depending upon Context.

The meaning remains identical.

Accessibility

Accessibility Tokens adapt the Design System while preserving understanding.

Examples.

`Runtime.Accessibility.Contrast`

`Runtime.Accessibility.Motion`

`Runtime.Accessibility.TextScale`

Accessibility Tokens should always strengthen understanding.

Never weaken hierarchy.

Device

Device Tokens describe runtime capabilities.

Examples.

`Runtime.Device.Mobile`

`Runtime.Device.TV`

`Runtime.Device.Desktop`

`Runtime.Device.Tablet`

These tokens should influence:

- presentation
- density
- expression

They should **never** redefine Composition.

Motion

Runtime Motion Tokens adapt behaviour according to environment.

Examples.

`Runtime.Motion.Standard`

`Runtime.Motion.Reduced`

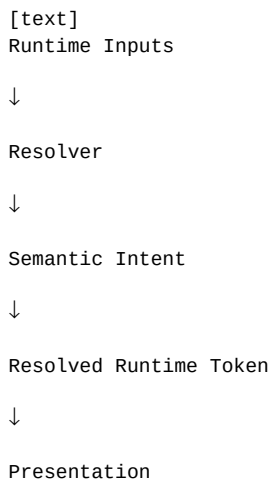
`Runtime.Motion.Disabled`

Movement philosophy remains unchanged.

Only implementation adapts.

Runtime Resolution

Every Runtime Token should be resolved through the same conceptual pipeline.



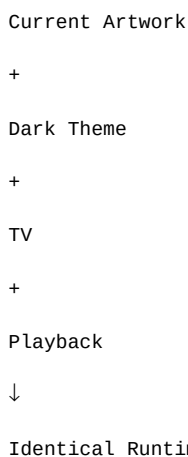
Components should never participate in runtime resolution.

Runtime Resolution Is Deterministic

Given identical inputs...

Runtime resolution should always produce identical outputs.

Example.



This determinism is critical for preserving consistency across devices.

Runtime Never Changes Meaning

Runtime Tokens may change:

- colour
- blur
- elevation
- opacity
- spacing

They must never change:

- hierarchy
- priority
- composition
- interaction

Those concepts belong to MDL.

Runtime exists only to improve expression.

Runtime Cache

Future implementations should assume Runtime Tokens are cacheable.

Stable runtime inputs should produce stable runtime outputs.

Example.

Artwork

↓

Atmosphere

↓

Cached

This significantly reduces unnecessary recomputation while preserving conceptual correctness.

Caching strategies belong to future runtime specifications.

Runtime And Components

Components consume Runtime Tokens exactly as they consume Semantic Tokens.

Example.

Tile.Hero

↓

Surface.Hero

↓

Runtime.Atmosphere.Primary

↓

Primitive Values

The component remains unaware of:

- artwork analysis
- accessibility
- theme generation

This separation dramatically simplifies implementation.

Plugins

Extensions never generate Runtime Tokens.

Plugins contribute:

- Information
- Relationships

The Runtime Resolver evaluates:

- current Composition
- current Focus
- artwork
- accessibility
- device

Runtime therefore remains entirely platform owned.

Good Examples

Artwork

↓

Atmosphere

↓

`Runtime.Atmosphere.Primary`

↓

Hero Surface

Accessibility

↓

Reduced Motion

↓

`Runtime.Motion.Reduced`

↓

Motion System

Device

↓

Television

↓

`Runtime.Device.TV`

↓

Presentation

Every example preserves semantic meaning.

Only implementation changes.

Anti-patterns

Runtime Components

`Runtime.Button.Primary`

Runtime should never know components.

Runtime Behaviour

`Runtime.CurrentPage`

Behaviour belongs to MDL.

Not Runtime.

Runtime Layout

`Runtime.LeftSidebar`

Layout should emerge from Composition.

Plugin Runtime

Extensions generating:

- colours
- hierarchy
- themes
- motion

Runtime ownership has been violated.

Runtime Model

[mermaid]
flowchart TD

```
World
World --> Composition
Composition --> SemanticTokens["Semantic Tokens"]
SemanticTokens["Semantic Tokens"] --> RuntimeResolver["Runtime Resolver"]
RuntimeResolver["Runtime Resolver"] --> RuntimeTokens["Runtime Tokens"]
RuntimeTokens["Runtime Tokens"] --> Presentation
```

Notice that Runtime exists after meaning has already been established.

Runtime refines expression.
It never defines understanding.

Relationship To Future Specifications

Future MDS specifications are expected to expand Runtime behaviour significantly.

Examples include:

- MDS-002 Colour System
- MDS-003 Material System
- MDS-005 Motion System
- MDS-006 Composition Engine

Those specifications define how Runtime Tokens are generated.

MDS-001 defines only where they belong within the architecture.

Summary

Runtime Tokens represent one of the most powerful capabilities of the Mosaic Design System.

They allow the platform to adapt continuously to:

- the user's World
- current artwork
- accessibility
- device
- environment

while preserving one stable conceptual architecture.

Understanding remains constant.

Presentation evolves naturally around it.

Review Status

Status

Draft

Next File

07-token-resolution.md

Token Resolution

Purpose

Previous chapters defined the layers of the Design Token Architecture.

This chapter defines **how those layers become usable design values at runtime**.

Token Resolution is responsible for transforming abstract design intent into concrete values without exposing implementation complexity to components.

Components should never ask:

"Which colour should I use?"

Instead they should ask:

"Which token represents my responsibility?"

The Design System resolves everything else.

Definition

Within MDS, **Token Resolution** is defined as:

The deterministic process by which an abstract design token is resolved into a platform-specific implementation value.

Resolution is an implementation process.

The Design Language remains unchanged throughout.

Why Resolution Exists

Consider the following component.

Hero Tile

Without token resolution, the component would need to understand:

- theme
- artwork
- accessibility
- platform
- device
- colour palette
- contrast
- motion

The component becomes responsible for far too many decisions.

Instead.

Hero Tile

↓

Surface.Hero

Everything else becomes the responsibility of the Design System.

One Direction

Token Resolution always flows in one direction.

[text]
Primitive

↓

Semantic

↓

Composition

↓

Runtime

↓

Platform

↓

Rendering

Tokens should never resolve backwards.

Presentation should never redefine semantic meaning.

Resolution Pipeline

Every token should pass through the same conceptual pipeline.

[mermaid]
flowchart TD

```
Primitive
Primitive --> Semantic
Semantic --> Composition
Composition --> Runtime
Runtime --> Platform
Platform --> Renderer
```

Every layer contributes one responsibility.

No layer duplicates another.

Resolution Inputs

Resolution may evaluate:

- current World
- current Focus
- current Context
- artwork
- accessibility
- user preferences
- device class
- platform theme

Importantly...

These inputs never change the semantic meaning of a token.

They only influence its implementation.

Deterministic Resolution

Given identical inputs...

Resolution should always produce identical outputs.

Example.

Surface.Hero

+

Current Artwork

+

Dark Theme

+

Desktop

↓

Resolved Surface

The same request should always produce the same result.

Deterministic behaviour is essential for:

- predictability
- testing
- caching
- accessibility

- consistency

Resolution Order

The Runtime Resolver should evaluate inputs in a consistent order.

[text]

1.

Primitive Values

↓

2.

Semantic Meaning

↓

3.

Composition Role

↓

4.

Runtime Inputs

↓

5.

Accessibility

↓

6.

Platform

↓

7.

Resolved Value

Earlier stages establish meaning.

Later stages refine implementation.

Resolution Never Changes Intent

One of the most important architectural guarantees of Mosaic is:

Resolution changes implementation.

Resolution never changes meaning.

Example.

Surface.Hero

Dark Mode.



Slate Background.

Light Mode.



White Background.

Artwork Mode.



Artwork-derived Acrylic.

The semantic meaning remains:

`Surface.Hero`

Meaning survives.

Implementation evolves.

Resolution Context

The Runtime Resolver should understand current context.

Example.

Playback



`Surface.Hero`

may resolve differently from:

Browsing



`Surface.Hero`

because runtime atmosphere changes.

The token itself remains identical.

Only resolution differs.

Resolution Priority

Multiple runtime influences may exist simultaneously.

Example.

Artwork

Accessibility

Dark Mode

Television

The Runtime Resolver should evaluate them using a defined priority.

Recommended conceptual priority.

Accessibility

↓

User Preferences

↓

Composition

↓

Artwork

↓

Device

↓

Platform Defaults

Accessibility should always take precedence over aesthetics.

Fallback Resolution

Every token must possess a valid fallback.

Poor.

Artwork Missing

↓

Failure

Preferred.

Artwork Missing

↓

Brand Tokens

↓

Resolved Value

Components should never receive unresolved tokens.

The Design System should always produce a meaningful result.

Lazy Resolution

Resolution should occur only when required.

Example.

Unused Token

↓

Not Resolved

The platform should avoid resolving values that will never be rendered.

This improves runtime efficiency while preserving identical behaviour.

Resolution Cache

Runtime resolution is expected to be cacheable.

Example.

Current Artwork

↓

Atmosphere

↓

Resolved Surface

↓

Cache

As long as the runtime inputs remain unchanged, the resolved value should remain stable.

Future Composition Engines may invalidate this cache when:

- Focus changes
- Context changes
- artwork changes
- accessibility changes

Resolution Is Invisible

Components should never know:

- how tokens were resolved
- where values originated
- whether runtime adaptation occurred

Components consume:

Resolved Tokens

Nothing else.

This dramatically simplifies component implementation.

Good Examples

Surface.Hero

↓

Runtime.Atmosphere

↓

Resolved Acrylic Surface

Text.Primary

↓

Accessibility

↓

Higher Contrast

Spacing.Section

↓

Television

↓

Expanded Physical Spacing

Every example preserves semantic meaning.

Only implementation changes.

Anti-patterns

Component Resolution

Component

↓

Determine Theme

↓

Determine Colours

Resolution responsibility has leaked.

Platform Resolution

Flutter

↓

Invent Semantic Meaning

Meaning belongs above implementation.

Runtime Mutation

Runtime

↓

Change Token Identity

Runtime should only resolve.

Never redefine.

Partial Resolution

Components receiving unresolved token chains.

Every rendered value should be fully resolved before presentation.

Resolution Model

[mermaid]
flowchart TD

```
Semantic
Semantic --> Composition
Composition --> Runtime
Runtime --> Resolution
Resolution --> PlatformTheme["Platform Theme"]
PlatformTheme["Platform Theme"] --> Rendering
```

Resolution exists to remove implementation complexity from the rest of the Design System.

Relationship To Future Specifications

Future specifications are expected to define:

- Runtime Resolver
- Theme Resolver
- Atmosphere Generator
- Accessibility Resolver
- Platform Adapters

These systems collectively implement the conceptual process defined by this chapter.

Summary

Token Resolution is the implementation bridge between design intent and rendered interface.

Its responsibility is to ensure that every component receives the correct implementation value while remaining completely unaware of:

- runtime adaptation

- artwork analysis
- accessibility
- platform differences
- device capabilities

The Design System owns complexity.

Components consume clarity.

Review Status

Status

Draft

Next File

08-token-inheritance.md

Token Inheritance

Purpose

One of the primary responsibilities of the Mosaic Design Token Architecture is allowing design decisions to evolve without requiring widespread implementation changes.

Token Inheritance provides this capability.

Instead of every component defining its own values, components inherit increasingly specialised design intent from higher layers of the architecture.

Inheritance therefore allows the Design System to express:

- consistency
- adaptability
- maintainability

through one coherent hierarchy.

Definition

Within MDS, **Token Inheritance** is defined as:

The process by which a token derives its meaning or implementation from another token while preserving a single responsibility at each layer.

Inheritance exists to preserve meaning.

Not duplicate values.

Why Inheritance Exists

Without inheritance, every component becomes responsible for resolving its own appearance.

Example.

Hero Tile

↓

Colour

↓

Spacing

↓

Radius

↓

Blur

↓

Elevation

Every component repeats identical decisions.

Instead.

Hero Tile

↓

Composition.Hero

↓

Surface.Hero

↓

Primitive Values

Responsibility becomes centralised.

Consistency naturally follows.

Inheritance Hierarchy

Inheritance always follows the same architectural direction.

[text]

Primitive

↓

Semantic

↓

Composition

↓

Component

↓

Runtime

↓

Platform

No layer should inherit directly from a lower layer.

Each layer inherits only the responsibilities it requires.

Primitive Inheritance

Primitive Tokens do not inherit.

They represent the physical foundation of the Design System.

Example.

Primitive.Space.16

This value is absolute.

Primitive Tokens terminate the inheritance chain.

Semantic Inheritance

Semantic Tokens inherit physical implementation.

Example.

`Surface.Primary`

↓

`Primitive.Colour.Slate.950`

The Semantic Token gains:

- colour
- contrast
- physical implementation

while preserving semantic meaning.

Composition Inheritance

Composition Tokens inherit semantic meaning.

Example.

`Composition.Hero`

↓

`Surface.Hero`

↓

`Text.Primary`

↓

`Elevation.Primary`

Composition therefore combines multiple semantic concepts into one compositional role.

Importantly...

It still does not know anything about components.

Component Inheritance

Components inherit compositional responsibilities.

Example.

`Hero Tile`

↓

Composition.Hero

↓

Semantic.Surface.Hero

↓

Primitive Values

The component consumes design intent.

It does not create it.

Runtime Inheritance

Runtime Tokens inherit the complete conceptual chain.

Example.

Runtime.Atmosphere.Primary

↓

Composition.Hero

↓

Surface.Hero

↓

Primitive Colour

Runtime then resolves implementation according to:

- artwork
- accessibility
- device
- user preferences

Meaning remains unchanged.

Platform Inheritance

Platform implementations inherit fully resolved Runtime Tokens.

Examples include:

- CSS Variables
- Flutter Theme
- SwiftUI Environment
- Compose Theme

Platform code should never reconstruct inheritance.

It should consume the completed result.

Multiple Inheritance

A token may inherit from multiple parent tokens.

Example.

Composition.Hero

↓

Surface.Hero

↓

Text.Primary

↓

Elevation.Primary

↓

Spacing.Section

The resulting role communicates:

- hierarchy
- material
- typography
- rhythm

without duplicating implementation.

Multiple inheritance should remain intentional and predictable.

Override Rules

Inheritance should support controlled overrides.

Permitted.

Runtime

↓

Accessibility Override

Not permitted.

Component

↓

Primitive Override

Bypassing semantic layers weakens the architecture.

Overrides should occur as high in the hierarchy as practical.

Local Overrides

Occasionally local overrides are necessary.

These should remain exceptional.

Before introducing an override ask:

1. Can an existing Semantic Token solve this? 2. Can Composition solve this? 3. Is Runtime the correct place? 4. Is a new token required?

Overrides should be the exception.

Not the default workflow.

Inheritance And Themes

Themes should never redefine semantic meaning.

Instead.

Surface.Primary

↓

Dark Theme

↓

Slate 950

Surface.Primary

↓

Light Theme

↓

Slate 50

The consuming component remains identical.

Only implementation changes.

Inheritance And Extensions

Extensions should inherit the complete Design System.

Plugins should consume:

- Semantic Tokens
- Composition Tokens

They should never redefine:

- Primitive values
- Runtime resolution

- Platform implementation

This guarantees ecosystem consistency.

Good Examples

Button

↓

Action.Primary

↓

Primitive Colour

Hero Tile

↓

Composition.Hero

↓

Surface.Hero

↓

Primitive Values

Timeline

↓

Composition.Supporting

↓

Surface.Secondary

↓

Primitive Values

Meaning accumulates.

Implementation remains centralised.

Anti-patterns

Primitive Consumption

Component

↓

Primitive

Semantic meaning has been bypassed.

Circular Inheritance

Semantic

↓

Component

↓

Semantic

Inheritance should always remain directional.

Component Overrides

Components redefining inherited values locally.

Consistency weakens.

Runtime Mutation

Runtime changing semantic identity.

Runtime resolves implementation.

It never changes meaning.

Inheritance Model

[mermaid]
flowchart TD

```
Primitive
Primitive --> Semantic
Semantic --> Composition
Composition --> Component
Component --> Runtime
Runtime --> Platform
Platform --> Rendering
```

Every layer contributes additional understanding.

No layer duplicates responsibilities owned elsewhere.

Relationship To Future Specifications

Future specifications should inherit from this architecture.

Examples include:

- Colour System
- Material System
- Typography
- Motion

- Component Library

None of these specifications should redefine inheritance.

They should extend it.

Litmus Test

Contributors should ask:

Can this change be made higher in the hierarchy?

If the answer is yes...

The change should almost always occur there.

Higher-level inheritance generally produces:

- fewer overrides
- stronger consistency
- easier maintenance

Summary

Token Inheritance allows Mosaic to express one coherent Design System across:

- multiple devices
- multiple themes
- runtime adaptation
- accessibility
- future implementations

Every layer inherits meaning from the layer above it while adding exactly one new responsibility.

This deliberate separation is what allows the Design System to evolve without losing conceptual integrity.

Review Status

Status

Draft

Next File

09-token-versioning.md

Token Versioning

Purpose

The Design Token Architecture is expected to evolve throughout the lifetime of Mosaic.

Primitive values will change.

Themes will evolve.

Materials will mature.

Accessibility requirements will improve.

Runtime capabilities will expand.

Token Versioning exists to ensure these changes can occur without unnecessarily breaking:

- applications
- extensions
- components
- tooling
- design assets

The goal is evolutionary change rather than disruptive replacement.

Definition

Within MDS, **Token Versioning** is defined as:

The controlled evolution of design tokens while preserving semantic stability and backwards compatibility wherever practical.

Versioning exists to protect intent.

Not implementation.

What Should Change?

Different layers possess different expected rates of change.

Layer	Change Frequency
Primitive Values	Occasionally
Semantic Tokens	Rarely
Composition Tokens	Very Rarely
Runtime Resolution	Frequently
Platform Implementation	Continuously

The higher the abstraction...

The more stable it should become.

Versioning Philosophy

The Mosaic Design System follows one fundamental rule.

<i>Values may evolve.</i>
<i>Meaning should remain stable.</i>

For example.

`Surface.Hero`

should continue representing the Hero surface for many years.

Its implementation may change from:

`Slate`

↓

`Acrylic`

↓

`Adaptive Material`

Applications consuming the token remain unchanged.

Semantic Stability

Semantic Tokens are considered part of the public design API.

Changing:

`Surface.Primary`

is significantly more expensive than changing:

`Primitive.Colour.Slate.950`

Consequently.

Semantic Token names should almost never change.

Instead:

- implementation evolves
- runtime evolves
- themes evolve

Meaning remains stable.

Version Numbers

The Design Token Architecture follows semantic versioning.

Major.Minor.Patch

Example.

1.0.0

Major.

Breaking semantic change.

Minor.

New compatible capability.

Patch.

Editorial or implementation correction.

Major Versions

Major versions are intentionally rare.

Examples include:

- semantic hierarchy redesign
- removal of token categories
- incompatible runtime model
- composition architecture changes

Major versions should normally require updates to downstream MDS specifications.

Minor Versions

Minor versions introduce:

- additional tokens
- additional categories
- optional runtime capabilities
- accessibility improvements
- theme expansion

Existing consumers should continue functioning without modification.

Patch Versions

Patch versions include:

- documentation improvements
- corrected mappings
- implementation fixes
- naming clarification

Patch versions should never change semantic behaviour.

Token Deprecation

Tokens should rarely be removed.

Instead they should enter a deprecation lifecycle.

Current

↓

Deprecated

↓

Superseded

↓

Removed

Deprecation should provide sufficient migration time for:

- applications
- extensions
- tooling
- documentation

Deprecation Metadata

Deprecated Tokens should include:

```
[yaml]
deprecated: true

replacement: Surface.Hero

removed_after: 2.0.0

reason: Consolidated Hero hierarchy
```

Consumers should receive clear migration guidance.

Never silent failure.

Aliasing

Temporary aliases may exist during migration.

Example.

`Surface.Featured`

↓

Alias

↓

`Surface.Hero`

Aliases should remain temporary.

Long-term duplication weakens the architecture.

Runtime Compatibility

Runtime evolution should remain backwards compatible whenever practical.

Example.

`Runtime.Atmosphere.Primary`

may internally evolve from:

`Static Colour`

to

`Artwork-derived Acrylic`

Consumers should remain unaware of the implementation change.

Extension Compatibility

Community extensions should consume:

- Semantic Tokens
- Composition Tokens

They should avoid consuming:

- Primitive Tokens
- Runtime implementation details

Doing so significantly reduces the likelihood of breaking changes.

Versioning Responsibilities

Layer	Responsibility
Primitive	Physical compatibility
Semantic	API stability
Composition	Behavioural stability
Runtime	Adaptive compatibility
Platform	Client compatibility

Every layer owns a different aspect of stability.

Migration Strategy

Preferred migration.

Old Token

↓

Alias

↓

New Token

↓

Deprecation Warning

↓

Removal

Avoid.

Old Token

↓

Deleted

Breaking changes should always be intentional.

Never surprising.

Changelog Requirements

Every token release should include:

- Added
- Changed
- Deprecated
- Removed

- Fixed

Each change should identify:

- affected token
- affected layer
- migration impact
- replacement

Token history is considered architectural documentation.

Anti-patterns

Renaming Semantic Tokens

Changing names without architectural justification.

Runtime Breaking Changes

Changing Runtime behaviour while preserving the same semantic meaning.

Runtime should evolve implementation.

Not intent.

Primitive API

Encouraging applications to consume Primitive Tokens directly.

Future migration becomes difficult.

Silent Removal

Deleting tokens without migration guidance.

Extensions become unnecessarily fragile.

Version Model

[mermaid]
flowchart TD

```
Primitive
Primitive --> Semantic
Semantic --> Composition
Composition --> Runtime
Runtime --> Platform
Semantic --> Stable
Primitive --> Evolves
Runtime --> Adapts
Platform --> Changes
```

Versioning should preserve architectural stability while allowing implementation to improve.

Relationship To Future Specifications

Future specifications should define:

- automated migration
- token validation
- compatibility tooling
- runtime schema versioning
- extension compatibility guarantees

These systems build upon the versioning philosophy established here.

Summary

Token Versioning protects one of the most valuable assets within the Mosaic Design System.

Its language.

Values will evolve.

Technology will evolve.

Presentation will evolve.

The meaning communicated by tokens should remain recognisable for many years.

That long-term semantic stability is what enables Mosaic to evolve confidently without fragmenting its design language.

Review Status

Status

Draft

Next File

10-extension-tokens.md

Extension Tokens

Purpose

Mosaic is fundamentally a platform.

Extensions are expected to become one of the primary ways the platform evolves over time.

The Design Token Architecture must therefore provide a mechanism through which extensions can participate in the Design System without fragmenting it.

This chapter defines that mechanism.

The objective is simple.

Every extension should feel native.

Not merely compatible.

Definition

Within MDS, **Extension Tokens** are defined as:

Tokens contributed by extensions that integrate into the Mosaic Design Token Architecture without redefining the platform's visual language.

Extension Tokens extend.

They never replace.

Why Extension Tokens Exist

Without a token architecture, extensions typically choose their own:

- colours
- spacing
- typography
- interaction
- hierarchy

Over time this produces several independent design languages inside one application.

Users no longer experience:

Mosaic

They experience:

Mosaic plus several unrelated interfaces.

Extension Tokens prevent this.

Platform Ownership

The core platform always owns:

- Primitive Tokens
- Semantic Tokens
- Composition Tokens
- Runtime Tokens

Extensions never redefine these layers.

Instead they consume them.

The platform remains the source of truth.

Extension Responsibilities

Extensions may contribute:

- new semantic concepts
- additional domain-specific meanings
- specialised capabilities

Examples include:

Manga

↓

Reading Progress

Audiobooks

↓

Listening Position

Calendar

↓

Upcoming Release

These concepts become part of the existing Composition rather than creating independent visual systems.

Information Before Presentation

Extension authors should think in information.

Not interface.

Poor.

Anime Extension

↓

Custom Hero

↓

Custom Timeline

↓

Custom Colours

Preferred.

Anime Extension

↓

Episode Release

↓

Studio

↓

Voice Actor

↓

Relationship

The platform determines how these concepts become interface.

Extension Token Namespace

Every extension should define its own namespace.

Example.

[text]
Extension

↓

Anime

↓

Episode

↓

Release

Or

[text]
Extension

↓

Books

↓

Chapter

↓

Progress

Namespaces prevent collisions while remaining readable.

Extension Categories

Extensions should contribute tokens only within clearly defined categories.

Examples include:

[text]
Extension

■■■ Domain

■■■ Capability

■■■ Information

■■■ Relationship

■■■ Expression Hint

■■■ Metadata

These categories intentionally avoid:

- colour
- spacing
- materials
- typography

Those remain platform concerns.

Expression Hints

An extension may suggest how information is naturally expressed.

Example.

Episode Countdown

↓

Timeline

This is an **Expression Hint**.

It is not an instruction.

The Composition Engine remains responsible for determining the final Expression.

If the current Context requires another Expression, the platform should ignore the hint.

Runtime Participation

Extensions participate in Runtime resolution indirectly.

Example.

Anime Extension



Episode Release



Runtime Context



Composition



Timeline



Presentation

The extension never resolves Runtime Tokens.

It merely enriches the information available to the Runtime Resolver.

Token Consumption

Extensions should consume the same Semantic Tokens as the core platform.

Examples.

Text.Primary

Surface.Secondary

Action.Primary

Border.Subtle

Using platform tokens ensures that extensions inherit:

- accessibility
- themes
- runtime atmosphere
- future redesigns

automatically.

Forbidden Responsibilities

Extensions should **not** define:

- Primitive Tokens
- Brand Tokens
- Runtime Tokens
- Material Tokens

- Motion Tokens
- Component Tokens

These remain exclusively owned by the platform.

Allowing extensions to redefine these layers would inevitably fragment the Design System.

Theme Compatibility

Because extensions consume Semantic Tokens rather than physical values:

Dark Mode.

↓

Works.

Light Mode.

↓

Works.

Artwork Atmosphere.

↓

Works.

Accessibility.

↓

Works.

The extension receives future improvements automatically.

No additional implementation is required.

Future Marketplace

The long-term goal of the Mosaic extension ecosystem is that users cannot visually distinguish:

Official functionality

from

Community functionality.

The Design Token Architecture is one of the primary mechanisms through which this consistency is achieved.

Users should recognise:

Mosaic

Not:

Several plugins sharing the same window.

Good Examples

Anime Extension

Contributes:

Episode Release

Studio

Opening Theme

Relationship

Consumes:

Surface.Primary

Text.Primary

Composition.Supporting

Books Extension

Contributes:

Reading Progress

Series Order

Bookmarks

Consumes:

Composition.Hero

Text.Secondary

Action.Primary

Music Extension

Contributes:

Current Track

Album

Concert

Consumes:

Surface.Hero

Composition.Primary

Runtime.Atmosphere

The visual language remains unified.

Anti-patterns

Brand Tokens

`Anime.Primary.Purple`

Extensions should not create competing brands.

Custom Themes

Extensions providing independent colour systems.

The platform loses visual coherence.

Runtime Generation

Extensions generating their own Runtime Tokens.

Runtime belongs exclusively to the platform.

Component Libraries

Extensions introducing:

- custom cards
- custom buttons
- custom layouts

The platform loses ownership of interface.

Extension Registration

Future runtime implementations should allow extensions to register token contributions declaratively.

Conceptually.

```
[yaml]
extension:
  id: anime

contributes:

  information:
    - episode.release
    - episode.runtime

  relationships:
    - adaptation
    - sequel

  expressions:
    - timeline
```

Notice that the extension contributes meaning.

Not presentation.

Token Resolution

Extension Tokens should participate in the standard resolution pipeline.

```
[mermaid]
flowchart TD
```

```
Extension
Extension --> Semantic
Semantic --> Composition
Composition --> Runtime
Runtime --> Presentation
```

Extensions never bypass the platform.

They enrich it.

Litmus Test

Extension authors should ask:

If Mosaic completely redesigned its interface tomorrow, would my extension still work?

If the answer is:

Yes.

The extension probably depends upon semantic architecture.

If the answer is:

No.

The extension probably depends upon implementation.

Summary

Extension Tokens allow Mosaic to scale into a large ecosystem without sacrificing a coherent design language.

Extensions contribute:

- knowledge
- capability
- relationships

The platform contributes:

- hierarchy
- composition
- runtime
- presentation

This separation ensures that every extension naturally becomes part of one unified Mosaic experience.

Review Status

Status

Draft

Next File

11-governance.md

Token Governance

Purpose

The Design Token Architecture is a shared contract between:

- Design
- Engineering
- Runtime Systems
- Extensions
- Tooling

Without governance, token systems naturally fragment.

Teams introduce duplicate tokens.

Semantic meaning drifts.

Components begin consuming primitive values.

Extensions invent competing token hierarchies.

Over time, the Design System becomes increasingly difficult to maintain.

This chapter defines how the Mosaic Design Token Architecture should evolve while preserving long-term consistency.

Governance Philosophy

Tokens should be treated as public APIs.

Once a token becomes part of the Design System, other systems begin depending upon it.

Changing a token therefore has architectural consequences.

Token governance exists to preserve:

- consistency
- compatibility
- maintainability
- predictability

rather than simply enforcing naming conventions.

The Token Contract

Every token published by the Design System forms part of the public design contract.

That contract guarantees:

- stable semantic meaning
- deterministic resolution
- documented inheritance
- predictable lifecycle

The contract intentionally does **not** guarantee:

- implementation values
- colour palettes
- runtime algorithms

These may evolve without breaking consumers.

Token Ownership

Each layer has a defined owner.

Layer	Owner
Primitive	Design Systems
Semantic	Design Systems
Composition	Design Systems
Runtime	Runtime Engine
Platform	Client Implementations

Ownership exists to preserve architectural integrity.

It does not prevent contribution.

Introducing New Tokens

Before introducing a token, contributors should answer the following questions.

Question One

Does an existing token already describe this intent?

Question Two

Is the proposed token semantic...
or implementation?

Question Three

Does the new token belong at the correct architectural layer?

Question Four

Will the token remain meaningful in five years?

Question Five

Would another contributor naturally discover this token?

If the answer to any question is "no", refinement should continue before implementation.

Token Creation Workflow

Every new token should follow the same lifecycle.

```
[mermaid]
flowchart TD
```

```
Need
Need --> ExistingTokenReview["Existing Token Review"]
ExistingTokenReview["Existing Token Review"] --> NewToken["New Token?"]
NewToken["New Token?"] -->|No| ReuseExisting["Reuse Existing"]
NewToken["New Token?"] -->|Yes| Proposal
Proposal --> Review
Review --> Documentation
Documentation --> Implementation
Implementation --> RuntimeValidation["Runtime Validation"]
```

The preferred outcome is always:

Reuse.

Not growth.

Layer Governance

Each layer follows slightly different governance rules.

Primitive

Growth should remain extremely slow.

Primitive Tokens should represent long-lived physical values.

Semantic

Semantic Tokens form the public Design API.

New Semantic Tokens require strong justification.

Composition

Composition Tokens affect runtime behaviour.

Changes require Design Systems review.

Runtime

Runtime Tokens may evolve more frequently.

However, they must never alter semantic intent.

Platform

Platform implementations should consume tokens.

They should not redefine them.

Token Reviews

Every proposed token should be reviewed against:

- MDL Vision
- Design Principles
- Mental Model
- Composition Model

The question should never be:

"Does this token work?"

Instead ask:

"Does this token strengthen the architecture?"

Duplicate Tokens

Duplicate semantic meaning is one of the greatest threats to long-term maintainability.

Poor.

Surface.Primary

Primary.Surface

Canvas.Primary

Three tokens.

One meaning.

Preferred.

Surface.Primary

One meaning.

One token.

One source of truth.

Token Drift

Token Drift occurs when:

- meaning changes
- implementation leaks upwards
- layers become blurred
- duplicate semantics appear
- component-specific tokens accumulate

Token Drift weakens the Design System significantly faster than visual inconsistency.

It should therefore be treated as architectural debt.

Token Debt

Examples include:

- duplicated Semantic Tokens
- unnecessary aliases
- Primitive Token consumption
- component-specific values
- undocumented Runtime Tokens

Token debt should be removed whenever practical.

Adding additional abstraction should never become easier than maintaining architectural clarity.

Extension Governance

Extensions may propose new Semantic Tokens.

They may not introduce:

- Primitive categories
- Runtime architecture
- Composition hierarchy

Core platform ownership remains preserved.

Extension proposals should demonstrate:

- reuse
- compatibility
- long-term usefulness

before acceptance.

Automated Validation

Future tooling should automatically validate:

- naming conventions
- hierarchy correctness
- inheritance
- duplicate semantics
- deprecated tokens
- unused tokens

Automated validation should supplement architectural review.

It should never replace it.

Governance Checklist

Every token proposal should satisfy:

- ☐ Correct architectural layer.
- ☐ Existing tokens evaluated.
- ☐ Semantic meaning documented.
- ☐ Runtime implications reviewed.
- ☐ Component impact assessed.
- ☐ Extension compatibility maintained.
- ☐ Documentation updated.

Success Criteria

The Token Architecture succeeds when:

- contributors naturally reuse existing tokens
- semantic names remain stable
- runtime adaptation remains invisible
- extensions integrate without fragmentation
- implementations remain framework independent

The strongest Design Token Architecture is one contributors rarely need to think about consciously.

They simply trust it.

Architectural Decisions

ADR	Decision
ADR-080	Design Tokens are treated as a public architectural contract.
ADR-081	Semantic stability has higher priority than implementation stability.
ADR-082	Reuse is preferred over introducing new tokens.
ADR-083	Token drift is considered architectural debt.

Review Status

Status

Draft

Next File

12-adrs.md

Architectural Decision Records

Purpose

The Architectural Decision Records (ADRs) contained within MDS-001 preserve the architectural reasoning behind the Design Token Architecture.

Where the MDL specifications established:

- Vision
- Principles
- Mental Model
- Interaction
- Composition

MDS-001 establishes how those concepts become machine-readable implementation.

These ADRs explain why the token architecture has been deliberately layered and why responsibility is separated between Primitive, Semantic, Composition and Runtime Tokens.

Future contributors should consult these ADRs before proposing structural changes to the Design System.

ADR Format

All Mosaic ADRs follow the standard structure.

[text]

ADR Number

Status

Context

Decision

Consequences

Alternatives Considered

Related Specifications

Each ADR records one architectural decision.

ADR-084

Title

Treat Design Tokens As Architectural Concepts

Status

Accepted

Context

Many Design Systems treat tokens as implementation variables.

This tightly couples tokens to frontend technologies.

Decision

Design Tokens become architectural concepts representing design intent rather than implementation values.

Consequences

Future client implementations remain interchangeable while sharing one common Design Language.

ADR-085

Title

Separate Primitive Tokens From Semantic Tokens

Status

Accepted

Context

Direct consumption of colour and spacing values weakens long-term maintainability.

Decision

Primitive Tokens represent physical values.

Semantic Tokens represent design meaning.

Consequences

Themes, runtime systems and accessibility improvements can evolve without changing component implementations.

ADR-086

Title

Introduce Composition Tokens

Status

Accepted

Context

Semantic Tokens alone cannot adequately describe compositional responsibilities.

Decision

Composition Tokens become an independent architectural layer.

Consequences

Runtime systems gain the ability to reorganise hierarchy without modifying component implementations.

ADR-087

Title

Introduce Runtime Tokens

Status

Accepted

Context

Traditional Design Systems assume static design values.

Mosaic requires adaptation based upon:

- artwork
- Focus
- Context
- accessibility
- device

Decision

Runtime Tokens become dynamically resolved implementation values.

Consequences

The platform becomes adaptive while preserving stable semantic architecture.

ADR-088

Title

Presentation Is Generated

Status

Accepted

Context

Different platforms require different implementation technologies.

Decision

Presentation becomes the generated output of the Design Token Architecture.

Consequences

Future platforms may consume identical Design Tokens while producing different implementation artefacts.

ADR-089

Title

Components Consume Meaning Rather Than Values

Status

Accepted

Context

Components directly consuming Primitive Tokens become tightly coupled to implementation.

Decision

Components consume Semantic and Composition Tokens.

Primitive Tokens remain hidden beneath the architecture.

Consequences

Component implementations become significantly easier to maintain and migrate.

ADR-090

Title

Runtime Never Changes Meaning

Status

Accepted

Context

Runtime adaptation introduces significant implementation flexibility.

Without architectural boundaries runtime could accidentally redefine semantic intent.

Decision

Runtime Tokens may alter implementation.

They must never alter semantic meaning.

Consequences

Artwork adaptation, accessibility and platform changes remain predictable.

ADR-091

Title

Extensions Participate Through Semantic Architecture

Status

Accepted

Context

Allowing extensions to introduce their own token hierarchies fragments product identity.

Decision

Extensions contribute information and consume existing Design Tokens.

The platform owns Runtime and Presentation.

Consequences

Community extensions naturally inherit future Design System improvements.

ADR-092

Title

Treat The Design Token Hierarchy As A Stable API

Status

Accepted

Context

Applications, tooling and extensions all depend upon token names.

Decision

The token hierarchy becomes a public architectural contract.

Semantic stability receives higher priority than implementation stability.

Consequences

Future redesigns become dramatically less disruptive.

ADR Relationships

[mermaid]
flowchart TD

ADR084["Design Intent"]

ADR085["Primitive / Semantic"]

ADR086["Composition"]

ADR087["Runtime"]

ADR088["Presentation"]

ADR089["Components"]

ADR090["Meaning"]

ADR091["Extensions"]

ADR092["Public API"]

ADR084 --> ADR085

ADR085 --> ADR086

ADR086 --> ADR087

ADR087 --> ADR088

ADR086 --> ADR089

ADR087 --> ADR090

ADR090 --> ADR091

ADR091 --> ADR092

Each decision reinforces a single architectural objective.

Meaning should remain stable while implementation remains flexible.

Future ADRs

Future Design Token ADRs are expected to formalise:

- Token Schema
- Token Registry
- Runtime Resolver
- Theme Generation
- Multi-Brand Architecture
- Token Validation
- Build Pipelines
- Design Tool Synchronisation
- SDK Code Generation

These intentionally remain outside the scope of MDS-001 Version 0.1.

ADR Governance

Changes to the Design Token Architecture should occur only when:

- semantic ambiguity exists,
- architectural layering proves insufficient,
- runtime evolution requires new abstraction,
- multiple specifications repeatedly conflict.

Implementation convenience is **not** sufficient justification for changing the architecture.

Summary

The ADRs contained within MDS-001 define one of the most important implementation boundaries within Mosaic.

Applications should never consume raw implementation values.

Instead they consume stable architectural meaning.

This separation allows the Design System to evolve continuously while preserving one coherent design language across every client, extension and future platform.

Review Status

Status

Draft

Next File

13-contributor-guidance.md

Contributor Guidance

Purpose

The Design Token Architecture is one of the few systems every Mosaic contributor will interact with.

Designers define tokens.

Engineers consume tokens.

Tooling generates tokens.

Extensions inherit tokens.

For this reason, MDS-001 establishes a common set of practices intended to preserve the long-term integrity of the Design System.

The objective is simple.

Every implementation should express the same design language without every contributor making the same design decisions repeatedly.

Think In Meaning

Before creating a token, contributors should ask:

What design decision am I trying to represent?

Not:

What value do I need?

Good.

`Surface.Hero`

Poor.

`#112233`

Values change.

Meaning survives.

Consume The Highest Layer

Always consume the highest architectural layer available.

Preferred.

`Component`

↓

`Composition Token`

↓

Semantic Token

↓

Primitive Token

Avoid.

Component

↓

Primitive Token

Skipping layers weakens maintainability.

Never Duplicate Meaning

Before introducing a new token ask:

- Does this meaning already exist?
- Could an existing Semantic Token express this?
- Could Composition communicate this instead?

One concept should normally correspond to one semantic token.

Multiple names for the same meaning create architectural drift.

Components Consume

Components should consume.

They should not decide.

Good.

Button

↓

Action.Primary

Poor.

Button

↓

Blue500

The Design System owns appearance.

Components own behaviour.

Runtime Is Invisible

Components should never ask:

- Which artwork is active?

- Which theme is loaded?
- Which device is this?

Instead they consume:

Resolved Tokens

Runtime adaptation is entirely the responsibility of the Runtime Resolver.

Prefer Composition Tokens

When implementing UI, contributors should normally begin with Composition Tokens.

Example.

Hero Tile

↓

Composition.Hero

rather than:

Hero Tile

↓

Surface.Primary

↓

Text.Primary

↓

Elevation.High

The Composition Token already communicates those responsibilities.

Higher layers reduce implementation complexity.

Extensions

Extension authors should think in capabilities rather than appearance.

Contribute:

- Information
- Relationships
- Expressions

Consume:

- Semantic Tokens
- Composition Tokens

Avoid introducing:

- colours

- spacing systems
- typography scales
- material definitions

The extension ecosystem should inherit the Mosaic visual language automatically.

Themes

Themes should never introduce new semantic meaning.

Good.

`Surface.Primary`

↓

Different Primitive Values

Poor.

`DarkSurfacePrimary`

`LightSurfacePrimary`

Themes change implementation.

Not architecture.

Runtime

Runtime Tokens should remain implementation details.

Contributors should almost never reference them directly unless working on:

- runtime systems
- theme generation
- composition engine
- platform adapters

Application code should normally consume Semantic or Composition Tokens instead.

Naming

Every token should answer one question.

Would another contributor immediately understand why this token exists?

If explanation is required...

The name should be reconsidered.

Names should communicate:

- purpose

- responsibility
- meaning

Never:

- colour
- framework
- implementation

Review Questions

Before introducing any token ask:

- Does this represent intent or implementation?
- Is this the correct architectural layer?
- Could an existing token solve the same problem?
- Will this still make sense after a redesign?
- Would another platform consume this token unchanged?

If any answer is uncertain, revisit the hierarchy before implementation.

Common Mistakes

Avoid:

Primitive Consumption

Components using raw colours and spacing.

Semantic Leakage

Embedding implementation into Semantic Tokens.

Runtime Leakage

Allowing Runtime concerns to appear in application code.

Component Tokens

Creating tokens that exist solely because a component exists.

Platform Thinking

Naming tokens after CSS, Flutter or SwiftUI concepts.

The Design System should outlive implementation technology.

Design Token Checklist

Every new token should satisfy the following.

- ☐ Represents one responsibility.
- ☐ Exists at the correct architectural layer.
- ☐ Communicates semantic intent.
- ☐ Does not duplicate existing meaning.
- ☐ Is implementation independent.
- ☐ Supports runtime adaptation.
- ☐ Preserves accessibility.
- ☐ Can be consumed across every Mosaic client.

Final Guidance

The Design Token Architecture exists to remove design decisions from application code.

Application developers should spend their time building:

- experiences
- capabilities
- interactions

not repeatedly choosing:

- colours
- spacing
- typography
- materials

If contributors find themselves repeatedly making visual decisions inside application code, they should stop and ask:

Should this decision become a Design Token instead?

When that instinct becomes natural, the Design System has succeeded.

Review Status

Status

Draft

Next File

glossary.md

Glossary

Purpose

This glossary defines the architectural terminology introduced by **MDS-001 — Design Token Architecture**.

Unlike previous MDL glossaries, this document focuses on implementation concepts rather than product concepts.

These definitions should be considered normative throughout every future MDS specification.

C

Composition Token

A token describing the compositional role of an element.

Composition Tokens communicate:

- hierarchy
- importance
- behavioural responsibility

They intentionally avoid communicating physical implementation.

Examples include:

- Hero
- Supporting
- Anchor
- Peripheral

Component Token

A token consumed directly by a reusable interface component.

Component Tokens inherit semantic meaning rather than defining it.

They should rarely reference Primitive Tokens directly.

I

Inheritance

The architectural process through which one token derives meaning from another.

Inheritance should always flow downward through the token hierarchy.

Meaning accumulates.

Responsibilities remain separated.

P

Platform Token

A platform-specific implementation generated from resolved Runtime Tokens.

Examples include:

- CSS variables
- Flutter ThemeData
- SwiftUI Environment values
- Compose theme objects

Platform Tokens are generated artefacts.

They are not authored manually.

Primitive Token

A token representing a measurable physical value.

Examples include:

- colour
- spacing
- blur
- elevation
- radius

Primitive Tokens intentionally contain no semantic meaning.

R

Resolution

The deterministic process through which a token becomes a concrete implementation value.

Resolution evaluates:

- Semantic meaning
- Composition
- Runtime inputs
- Accessibility
- Platform

before producing a renderable value.

Runtime Token

A dynamically resolved token representing the current runtime environment.

Examples include:

- Atmosphere
- Device
- Accessibility
- Current Focus

Runtime Tokens never redefine semantic meaning.

They only refine implementation.

S

Semantic Token

A token representing design intent independently from implementation.

Examples include:

- Surface.Primary
- Text.Secondary
- Action.Primary
- Border.Subtle

Applications should consume Semantic Tokens rather than Primitive Tokens whenever possible.

T

Theme

A mapping between Semantic Tokens and Primitive Tokens.

Themes alter implementation.

They should never alter semantic intent.

Token

The smallest implementation unit capable of expressing a design decision independently from implementation technology.

Tokens communicate intent before values.

Token Hierarchy

The ordered architectural structure of the Design Token Architecture.

Primitive

↓

Semantic

↓

Composition

↓

Component

↓

Runtime

↓

Presentation

Each layer contributes one responsibility.

Token Resolution

The runtime process responsible for converting abstract tokens into platform-specific implementation values.

Resolution should remain deterministic and invisible to application code.

Token Drift

The gradual weakening of the Design Token Architecture through:

- duplicate semantics
- implementation leakage
- inconsistent naming
- incorrect layering

Token Drift is considered architectural debt.

Token Debt

Accumulated architectural complexity caused by poor token design.

Examples include:

- duplicated tokens
- unnecessary aliases
- Primitive consumption
- undocumented Runtime Tokens

Token Debt should be reduced continuously.

Cross References

Specification	Primary Concepts
MDL-001 Vision	Product Philosophy
MDL-002 Principles	Design Intent
MDL-003 Mental Model	World, Information
MDL-004 Interaction Model	Behaviour
MDL-005 Composition Model	Hierarchy, Composition
MDS-002 Material System	Semantic Consumption
MDS-006 Composition Engine	Runtime Resolution

Terminology Rules

Future contributors should:

- describe meaning before values
- describe semantics before implementation
- consume higher-level tokens whenever practical
- avoid platform-specific terminology within architectural specifications

The Design Token Architecture should remain understandable independently from any implementation technology.

Review Status

Status

Draft

Next File

references.md

References

Purpose

This document records the architectural influences, design theories and implementation concepts that informed **MDS-001 — Design Token Architecture**.

Unlike implementation documentation, these references exist to explain the architectural reasoning behind the Design Token Architecture rather than prescribe implementation details.

The Mosaic Design System intentionally synthesises established Design System practices with the adaptive, runtime-first philosophy established throughout the MDL specifications.

Reading Order

Contributors should approach references in the following order.

1. MDL Specifications 2. Design Token Architecture 3. Design Systems 4. Runtime Design 5. Cross-Platform Architecture 6. Implementation Specifications

The Mosaic Design Language remains the authoritative source.

External references provide context rather than authority.

Internal References

MDL-001 — Vision

Provides:

- Product philosophy
- Immersion
- Companion model
- Long-term product identity

Every token should ultimately reinforce the experience defined by the Vision.

MDL-002 — Principles

Provides:

- Design intent
- Behavioural priorities
- Design governance

Semantic Tokens should communicate these principles through implementation.

MDL-003 — Mental Model

Provides:

- World
- Focus
- Context
- Information
- Relationships

Runtime Tokens are intentionally designed to adapt these concepts rather than replace them.

MDL-004 — Interaction Model

Provides:

- Behaviour
- Continuity
- Adaptive evolution
- Interaction States

Runtime Tokens should reinforce behavioural consistency rather than introduce alternative interaction models.

MDL-005 — Composition Model

Provides:

- Hero
- Hierarchy
- Priority
- Density
- Composition

Composition Tokens are the implementation of these conceptual ideas.

Future Specifications

The following specifications depend directly upon MDS-001.

- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography
- MDS-005 Motion System
- MDS-006 Composition Engine

- MDS-007 Tile Framework
- MDS-008 Component Library

These specifications should consume the architecture defined here rather than redefining it.

Design Tokens

The architecture of MDS-001 draws inspiration from modern token systems that distinguish between:

- physical values
- semantic meaning
- implementation

Mosaic extends this approach by introducing:

- Composition Tokens
- Runtime Tokens

allowing the Design System to adapt continuously while preserving semantic stability.

Runtime Design

Unlike traditional Design Systems, Mosaic assumes that:

- artwork changes
- Context changes
- Focus changes
- accessibility changes
- device capabilities change

The Design Token Architecture therefore treats runtime adaptation as a first-class architectural concern rather than an implementation detail.

Cross-Platform Design

MDS intentionally separates:

Meaning

↓

Runtime

↓

Platform

↓

Rendering

This separation allows identical Design Tokens to generate implementations for:

- Web
- Flutter
- SwiftUI
- Jetpack Compose
- Future clients

without changing the architecture.

Design Systems

The organisation of the Mosaic Design System was influenced by mature Design Systems that distinguish:

- Foundations
- Tokens
- Components
- Patterns
- Guidelines

Mosaic intentionally extends this model through:

- runtime adaptation
- composition solving
- artwork-driven atmosphere
- semantic hierarchy

These concepts distinguish Mosaic from traditional static Design Systems.

Software Architecture

Several software architecture concepts influenced MDS-001.

Examples include:

- Separation of Concerns
- Layered Architecture
- Domain-Driven Design
- Documentation as Code
- Architecture Decision Records
- Platform Abstraction

These concepts informed documentation organisation rather than implementation technology.

Mosaic-Specific Influences

The Design Token Architecture emerged directly from the preceding MDL specifications.

Key architectural discoveries include:

- Meaning should precede implementation.
- Components should consume intent rather than values.
- Runtime should adapt implementation rather than semantics.
- Composition should remain independent from presentation.
- Extensions should inherit the Design System rather than redefining it.

These ideas collectively define the Mosaic Design System.

Relationship To The Runtime

Future runtime systems are expected to implement concepts such as:

- Atmosphere generation
- Adaptive materials
- Theme resolution
- Device adaptation
- Accessibility adaptation

MDS-001 intentionally defines the architectural boundaries within which those systems operate.

Implementation details belong to future specifications.

Normative References

Required reading before contributing to MDS-001.

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model

These documents collectively define the conceptual foundation upon which the Design Token Architecture is built.

Informative References

Future contributors may also wish to review:

- MDS-002 Colour System
- MDS-003 Material System
- MDS-006 Composition Engine
- MDS-008 Component Library

These specifications implement the token architecture defined within MDS-001.

Living Document

This reference list should remain intentionally concise.

References should only be introduced when they materially influence:

- architectural layering
- runtime adaptation
- semantic modelling
- implementation boundaries

The purpose of this document is to preserve architectural reasoning rather than provide an exhaustive catalogue of design literature.

Completion

This concludes **MDS-001 — Design Token Architecture**.

The next specification in the Mosaic Design System is:

MDS-002 — Colour System

Where MDS-001 defines **how design intent becomes machine-readable**, MDS-002 defines **how colour communicates meaning**.

It formalises:

- the Mosaic brand palette
- semantic colour architecture
- adaptive atmosphere
- artwork-derived colour generation
- accessibility-aware colour resolution
- light and dark themes
- runtime colour synthesis

It is here that the distinctive visual identity of Mosaic begins to emerge from the architectural foundations established by the MDL and MDS specifications.